Задание 1-3 (Рабочая тетрадь 1)

Задание 1: Автоматизация языковой конверсии с использованием транслятора. Напишите программу, которая автоматически переводит небольшой фрагмент кода с одного языка программирования на другой. Выберите два языка, например, Python и Java. Программа должна принимать на вход исходный код на Python и генерировать эквивалентный код на Java. Для этого можно использовать существующие библиотеки или инструменты для трансляции, такие как ANTLR или другие аналогичные инструменты.

Для решения текущей задачи был выбран путь написания транслятора основоного на LLM модели запущенной локально. В результате тестов была подобрана подходящая под данную задачу модель **qwen3-coder:30b**.

Установленные нейросети и конфигурация оборудования:

Конфигурация системы: `Процессор: Ryzen 7 7700x, Видеокарта: RTX 3070, Оперативная память: 32GB ddr5, Накопитель: SSD M.2 Kingston 1024gb.`

После установки Ollama была прозведена установка нейросетей для тестов и дальшейшей работы

```
ollama pull qwen3-coder:30b
```

```
ollama pull deepseek-coder-v2
```

```
ollama pull qwen2.5-coder:7b
```

после установки проверка наличия нейросетей на устройстве осуществялется с помошью команды:

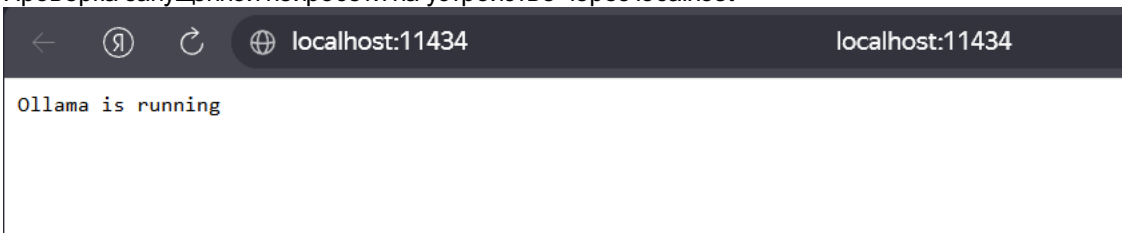
```
ollama list
```

```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindows

PS C:\Users\Satan> ollama list
NAME                        ID              SIZE      MODIFIED
qwen3-coder:30b             06c1097efce0    18 GB     5 days ago
deepseek-coder-v2:latest    63fb193b3a9b    8.9 GB    5 days ago
qwen2.5-coder:7b            dae161e27b0e    4.7 GB    6 days ago
PS C:\Users\Satan>
```

После чего осуществляем процесс запуска выбранной по итогу нами модели на локальном устройстве

```
ollama run qwen3-coder:30b
```

Проверка запущенной нейросети на устройстве через localhost



Пайплайн процесса трансляции кода

1. **Приём входных данных**
2. Параметры: `source_lang`, `target_lang`, путь к файлу с исходным кодом.

3. Считывание исходного кода в строку `source_code`.

4. **Инициализация транслятора**

5. Создание экземпляра `UniversalTranslator` с моделью LLM (например, `qwen2.5-coder:7b`), адресом Ollama, параметрами `temperature=0`, `num_ctx=8192`, `max_attempts=3`.

6. **Цикл попыток (attempt = 1..3)**
   3.1. **Формирование промпта**

   - Базовый текст с жёсткими правилами (сохранять структуру, запрет рефакторинга, допустимые действия).
   - Вставка `source_lang`, `target_lang`, исходного кода.
   - Если `error_feedback` не пустая (со второй попытки) — добавить секцию с предыдущей ошибкой компиляции/синтаксиса.

3.2. **Вызов LLM**
- HTTP `POST` запрос к `http://localhost:11434/api/generate` с JSON:
`{"model": "...", "prompt": "...", "stream": false, "options": {"temperature": 0}}`
- Получение ответа `response["response"]`.

3.3. **Извлечение кода**
- Удаление маркеров Markdown: шаблоны `target_lang ...`, `...`, `` ... - Если нет блоков — возвращается вся строка ответа как есть. - Результат — строка`translated`.

3.4. **Синтаксическая валидация**
- В зависимости от `target_lang`:
- **Python** → `compile(translated, ...)`
- **Java** → запись во временный `.java` файл → `javac <файл>` (без `-d`)
- **C** → `gcc -fsyntax-only`
- **C++** → `g++ -fsyntax-only`
- **JavaScript** → `node --check -e`
- Прочие → пропуск (валидация = успех)
- Результат: `(valid, error_message)`

3.5. **Ветвление**
- Если `valid == True` → выход из цикла, возврат `translated`.
- Если `valid == False` → сохранить `error_message` в `last_error`, присвоить `current_code = translated`, увеличить счётчик попыток, перейти к 3.1.

1. **Завершение**
2. Если после 3 попыток нет валидного кода → выброс исключения `RuntimeError`.
3. При успехе — запись результата в файл `output.{target_lang}` и вывод на экран.

Для использования транслятора необходимо перейти в директорию с исполнительным файлом транслятора:

```
cd Prac1\PR1_1\
```

После чего управление транслятором осуществвляется через командную строку в соответствии с данным синтаксисом:

python universal_translator.py <исходный_язык> <целевой_язык> <файл_с_исходным_кодом>

Примеры:

```
python universal_translator.py python java sieve.py
```

```
python universal_translator.py python c factorial.py
```

```
python universal_translator.py java python Main.java
```

```
python universal_translator.py python javascript script.py
```

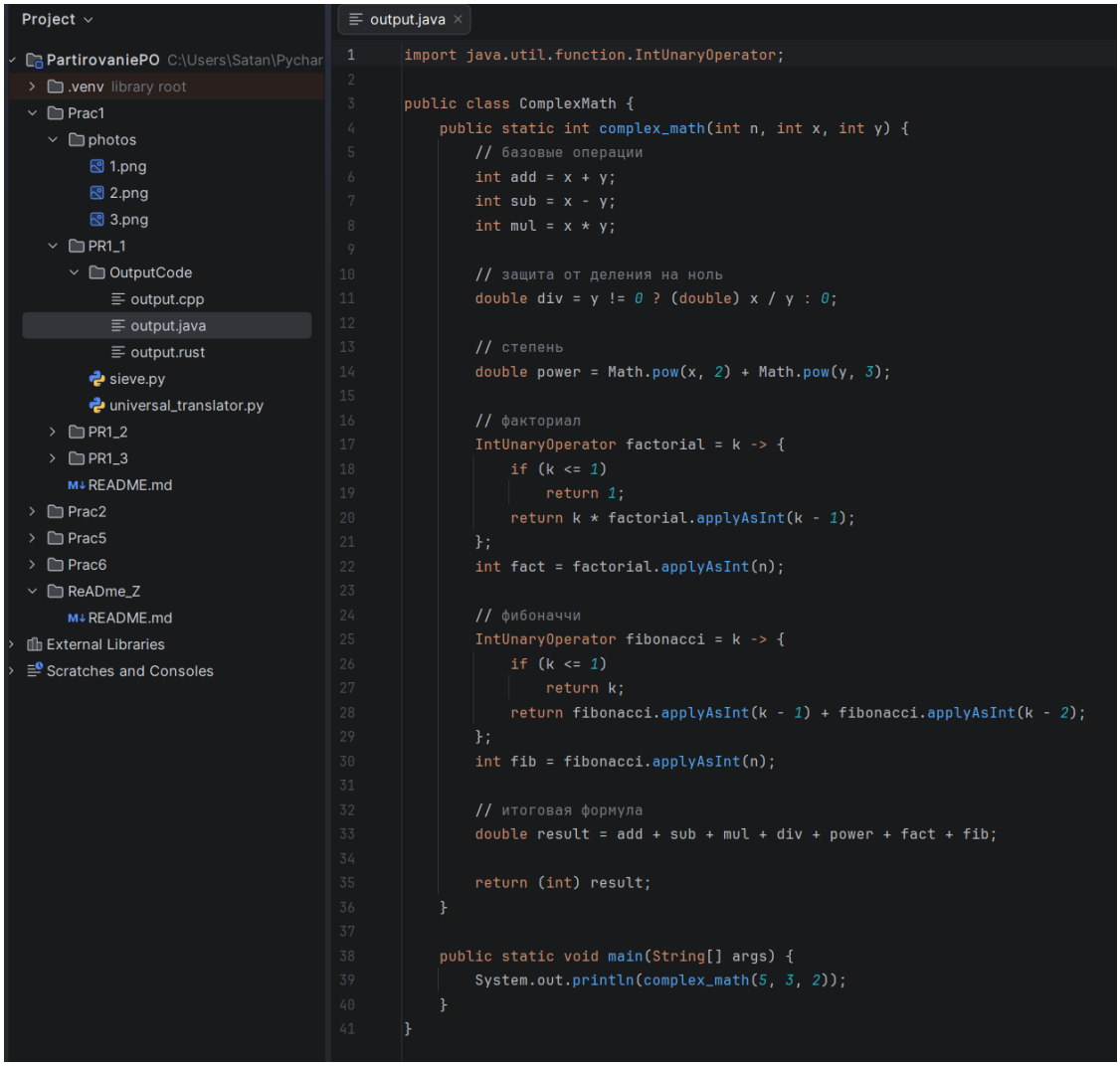
В результате успешной работы программы код будет сохранен в папку **OutputCode** и продублирован в консоль. Пример работы программы:

```
python universal_translator.py python java sieve.py
```

Дублирование в консоль:

```
Попытка 1 успешна!

 ----Переведенный код----

import java.util.function.IntUnaryOperator;

public class ComplexMath {
    public static int complex_math(int n, int x, int y) {
        // базовые операции
        int add = x + y;
        int sub = x - y;
        int mul = x * y;

        // защита от деления на ноль
        double div = y != 0 ? (double) x / y : 0;

        // степень
        double power = Math.pow(x, 2) + Math.pow(y, 3);

        // факториал
        IntUnaryOperator factorial = k -> {
            if (k <= 1)
                return 1;
            return k * factorial.applyAsInt(k - 1);
        };
        int fact = factorial.applyAsInt(n);

        // фибоначчи
        IntUnaryOperator fibonacci = k -> {
            if (k <= 1)
                return k;
            return fibonacci.applyAsInt(k - 1) + fibonacci.applyAsInt(k - 2);
        };
        int fib = fibonacci.applyAsInt(n);

        // итоговая формула
        double result = add + sub + mul + div + power + fact + fib;

        return (int) result;
    }

    public static void main(String[] args) {
        System.out.println(complex_math(5, 3, 2));
    }
}

Сохранено в output.java
```

Итоговый файл:



Листинг кода:

Исходный код транслятора: universal_translator.py

Исходный код входного файла (показанного выше): sieve.py

Выходной код транслятора: output.java

Задание 2: Разработать кроссплатформенное приложение, которое будет работать как на Python, так и на Java. Приложение должно выполнять базовую функцию, например, вычисление факториала числа или обработку текста. Для реализации кроссплатформенности, вам потребуется написать код на одном языке (например, Python) и затем перевести его на другой язык (например, Java) с использованием инструментов для языковой конверсии.

Для решения данного задания было написанно приложение на языке Python для обработки текста

Листинг кода прилогается: text_processor.py

Работа кода на данном примере текста: test.txt

```
(.venv) PS C:\Users\Satan\PycharmProjects\PartirovaniePO\Prac1\PR1_2> python text_processor.py test.txt
Файл: test.txt
Символов: 6043
Слов: 990
Предложений: 75

--- Первые 200 символов текста ---
 На европейские события в 1854 году

 С чего взялась всесветная беда?
 Кто виноват, кто первый начинает?
 Народ вы умный, всякой это знает,
 Да славушка пошла об вас худа!
 Уж лучше бы в покое дома жи...
```

Далее с помощью написанного транслятора кода реализуем перевод кода на язык Java с помощью данной команды:

```
python universal_translator.py python java text_processor.py
```

В результате мы получили программу на языке java output.java

Задание 3: Написать программу, которая автоматически переводит простые математические функции, написанные на Python, на Java. Программа должна принимать на вход исходный код на Python, анализировать его и генерировать эквивалентный код на Java. Программа должна поддерживать базовые математические операции, такие как сложение, вычитание, умножение, деление, возведение в степень, а также простые функции, такие как вычисление факториала или ряда Фибоначчи.

Опять же для реализации данной задачи был выбран путь использования уже существующего инструмента universal_translator.py описанного выше.

Для реализации задачи нам нужно было написать код содержащий в себе математические функции разной сложности для базового языка был выбран питон в результате чего был написанн данный код TestFib-Rec.py

Работа данного кода выглядит следующим образом:

```
def complex_math(n, x, y):
    ...
    ...
    ...


    result = add + sub + mul + div + power + fact + fib
    return result
```

Из кода понятно что для начала мы передает 3-и числовые переменные в функцию **complex_math** после чего получаем результат который представляет ссобой сумму всех математических операций произведенных над числами.

Проверим выходной код после транслятора

Для проверки выбрал язык **Rust** и **Java**

Вывод программы на языке **Java**: output.java

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2022.2.3\lib\idea_rt.jar=52684:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2022.2.3\bin" -Dfile.
155

Process finished with exit code 0
```

Вывод программы на языке **Rust**: output.rust

Rust

```rust
fn complex_math(n: i32, x: f64, y: f64) -> f64 {
    // базовые операции
    let add = x + y;
    let sub = x - y;
    let mul = x * y;

    // защита от деления на ноль
    let div = if y != 0.0 { x / y } else { 0.0 };

    // степень
    let power = x.powi(2) + y.powi(3);

    // факториал
    fn factorial(k: i32) -> i32 {
        if k <= 1 {
            1
        } else {
            k * factorial(k - 1)
        }
    }

    let fact = factorial(n);

    // фибоначчи
    fn fibonacci(k: i32) -> i32 {
        if k <= 1 {
            k
        } else {
            fibonacci(k - 1) + fibonacci(k - 2)
        }
    }

    let fib = fibonacci(n);

    // итоговая формула
    let result = add + sub + mul + div + power as f64 + fact as f64 + fib as f64;

    return(result)
}


fn main() {
    let result = complex_math(5, 6.0, 8.0);
```

Run

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	2.06 Mb

Your Output

```
Результат: 733.75
```